

FULL STACK DEVELOPMENT

FASE 5 | AULA 06 -

CLEAN ARCHITECTURE - DESIGN PRINCIPLES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

Nas últimas aulas falamos muito sobre os componentes e, agora, vamos explorá-los detalhadamente: o que são, como funcionam, como podem ser usados e como pensar em componentes quando estamos construindo nossa Arquitetura. Também entenderemos algumas ideias por trás de uma boa arquitetura de software, e como as ideias de Arquitetura Limpa nos ajudam a tomar melhores decisões.



HANDS ON

Nos vídeos a seguir vamos discutir e aplicar algumas ideias para componentizar uma solução criada como um grande grupo de código unitário.

Enquanto fazemos isso, vamos entender também como a componentização ajuda a entender o software e seus limites e responsabilidades de forma mais clara. Vamos lá?!



SAIBA MAIS

Desde o início dessa disciplina, falamos sobre o uso de componentes para o desenho de nossa Arquitetura, como cada componente faz parte de cada camada, como os componentes interagem entre si. Mas agora que entendemos como funcionam as camadas da Arquitetura Limpa, precisamos nos aprofundar na ideia dos componentes da Arquitetura, entendendo a origem do pensamento de componentes, como eles precisam ser definidos e porque o uso de componentes nos ajuda a resolver os problemas de software relacionados à complexidade.

Neste momento, não queremos mais pensar em códigos ou em como implementar. O que queremos é compreender o processo de detalhar a solução usando componentes de arquitetura, que podem ser posteriormente implementados. Mesmo que uma boa arquitetura leve em conta as restrições do contexto tecnológico de implementação (plataforma, linguagem de programação e integrações obrigatórias), entender a arquitetura como um desenho de organização interna do produto nos ajuda a aprender a tomar decisões melhores.

O que são componentes?

Os componentes de software são unidades de código que tem responsabilidade bem definida, e são agrupados e organizados para resolução de problemas. O termo “responsabilidade bem definida” é digno de atenção redobrada, porque os componentes de um software precisam saber fazer coisas simples, mas com total competência. Por exemplo, um componente de acesso a um servidor de banco de dados precisa saber fazer toda a comunicação com este servidor de forma totalmente competente, para que possa ser utilizado de forma confiável. Implementar apenas parte da comunicação pode ser útil, mas talvez para uma quantidade menor de situações.

Um componente de software bem construído tem interfaces de uso bem definidas. Voltando ao exemplo do componente de acesso ao SGBD, se as interfaces de uso (“abrir conexão”, “rodar comando” e “rodar pesquisa”) não forem explícitas, de forma que o nome da operação corresponda à operação, ou se não forem utilizáveis

de forma simples, o componente não será usado, porque ele causa mais problemas do que entrega solução.

Essa possibilidade de usar componentes e os conectar por meio de suas interfaces permite construir soluções complexas a partir de componentes menos complexos que a solução total. E assim podemos pensar em uma arquitetura que atende à nossa necessidade, usando os componentes em vez de criar um grande monolito complexo que implementa tudo.

Arquitetura monolítica

Uma arquitetura monolítica é aquela que centraliza, numa única unidade de código, todas as responsabilidades do software. É um modo de pensar a solução onde o foco é criar um “super software”, que é altamente integrado internamente, e que agrupa dentro de si as diversas responsabilidades da solução: interface com usuário, acesso a recursos externos, regras de negócio, configurações etc.

Uma arquitetura monolítica não é intrinsecamente ruim. Para soluções pequenas e/ou com um conjunto muito restrito de interfaces, ou que são desenvolvidas para um caso de uso extremamente específico, como software embarcado em microdispositivos ou nanodispositivos, a arquitetura monolítica é adequada, porque o software tem uma complexidade muito específica, e as interfaces são adaptadas ao dispositivo.

As arquiteturas monolíticas são um problema quando implementam diversas responsabilidades em um único componente, ou delegam responsabilidades para alguns poucos componentes que são altamente especializados, e no fim, todas implementações ficam centralizadas em um único componente. Manter esta solução funcionando, seja alterando o software ou até mesmo mantendo em operação, se torna uma tarefa complicada, aumentando o custo de manutenção de acordo com o tempo de existência da solução.

Componentização e responsabilidade

A componentização da solução nos ajuda a entender como podemos criar a solução em pedaços. Cada um desses pedaços é especializado em sua

responsabilidade e, ao mesmo tempo, é preparado para trabalhar bem quando for parte de uma solução maior. É como uma parte de um carro ou de uma máquina: sozinha, a parte é funcional, mas só é útil quando conectada a algo maior.

No caso da arquitetura de software, a criação dos componentes deve seguir sempre a regra da responsabilidade mínima: o componente deve ser capaz de implementar o mínimo possível para que seja útil dentro de um contexto; o que existir a mais deve ser segregado, e não deve fazer algo de forma incompleta.

Vamos lembrar da camada de entidades da Arquitetura Limpa: quando criamos um elemento de código que representa Estudante, o fazemos pensando (1) se este componente é útil, ou seja, se ele é bem definido em seus limites e responsabilidade, e (2), se ele é competente dentro do que é necessário. Em nosso caso, implementamos o elemento como uma classe e daí colocamos dentro da classe a definição dos campos necessários para caracterizar um Estudante de forma inequívoca, e implementamos a validação dos dados. E se definimos esta classe Estudante como herdando de uma classe Pessoa, conseguimos isolar bem a responsabilidade: o contexto de um Estudante é diferente de uma Pessoa, sendo possivelmente uma extensão dessa.

Reutilização, bibliotecas, repositórios, frameworks e código aberto

Quando estamos construindo nossa arquitetura, podemos usar inúmeros componentes, usar o quanto seja necessário para que nossa solução funcione. Não vamos usar todos os componentes disponíveis apenas porque eles existem, sem motivo lógico. Longe disso. Vamos usar a quantidade adequada, que pode ser grande ou pequena.

Mas não precisamos criar e recriar todos estes componentes para cada nova arquitetura ou novo projeto. Outra vantagem de criar bons componentes é para conseguir reutilizar cada um destes componentes em novos projetos; por exemplo, se a camada de conexão à uma API ou a um serviço funciona bem para um projeto, e ela foi criada com as boas práticas de isolamento e, portanto, é suficientemente desacoplada, podemos reutilizar em outro projeto. Isso é especialmente útil quando trabalhamos com um grande número de projetos, seja como indivíduo ou em um grupo de pessoas, porque temos disponível um grande número de componentes que

podemos utilizar em nossa arquitetura. Existem até mesmo repositórios de componentes em algumas organizações, que servem para documentar e disponibilizar componentes e soluções testadas e aprovadas para uso.

Usualmente, os componentes são organizados em bibliotecas, de acordo com o seu contexto de uso. Componentes relacionados a uma regra de negócio específica são agrupados em uma biblioteca e, então, quando precisamos usar alguns desses componentes, podemos obter a biblioteca inteira; e isso não quer dizer que vamos usar a biblioteca inteira, mas sim que vamos usar apenas os componentes necessários; entretanto, para a organização, dos componentes precisamos obter a biblioteca inteira.

As bibliotecas de código e de componentes existem há décadas, e têm sido o modo mais disseminado para compartilhamento de códigos. Nos últimos anos, com a 'universalização' do uso da internet dentro da área de desenvolvimento, as soluções de controle de código descentralizadas (como o git para controle de código e os sites como github.com e gitlab.com para armazenamento de código) permitiram a possibilidade de compartilhar desde pequenos pedaços de código até componentes completos para uso em nossas arquiteturas. Dessa forma, podemos usar estes componentes e até adaptar para nossa necessidade, uma vez que estes são distribuídos de forma aberta. É comum, hoje em dia, os pacotes de ferramenta (toolkits) das linguagens de programação incluírem ferramentas para buscar e copiar bibliotecas de componentes para usar em nossos projetos, como o npm (para Javascript/Typescript) e o pip (para Python).

Junto com a distribuição dos componentes, houve também um crescimento da distribuição de um tipo especial de software, os **frameworks**. Um framework é uma estrutura de código que é utilizada para agilizar a construção de um ou mais componentes, como se fosse uma casa pré-fabricada; dessa forma, é possível entregar um componente mais robusto, porque a base dele já é conhecida e testada. O principal uso dos frameworks hoje está na construção de APIs RESTful, com diversas soluções existentes e com algumas delas se tornando referências de mercado, como o Spring (Java) e o Express (Javascript).

A grande maioria dessas soluções pode ser usada sem custo, por serem distribuídas sob diferentes licenças de código aberto. Assim, construir uma arquitetura de software atualmente também envolve entender o que será construído, o que será

reutilizado a partir de soluções existentes dentro da organização e o que será aproveitado entre os componentes disponíveis para uso livre.

Qual a importância de pensar em componentes?

Com um panorama geral do que são componentes, podemos agora pensar em componentes, e como integrá-los para a criação de nossa arquitetura. Aqui, não temos uma receita de uso, ou um guia de implementação que nos permite tomar a decisão de forma automática. Mas temos alguns conceitos que podem nos ajudar a tomar melhores decisões.

Isolamento de responsabilidade

Como já falamos exaustivamente desde o começo, o pensamento de componentes nos permite pensar em **“o que isto deve fazer?”**. Esta pergunta nos força a pensar nas responsabilidades do componente dentro do contexto da arquitetura que estamos construindo, e também entender se este componente é adequado para o trabalho que ele precisa executar.

Com isso em mente, a regra do isolamento deve ser: cada componente com sua responsabilidade, e componentes não devem sobrepor ou compartilhar as responsabilidades, porque se dois componentes fazem a mesma coisa, como decidir qual usar? Essa questão não pode existir em um desenho de Arquitetura Limpa, porque os componentes têm suas camadas bem definidas, suas interfaces e adaptadores para facilitar a comunicação.

Composição e desacoplamento

Esse isolamento de responsabilidade dos componentes tem um objetivo: fazer com que seja possível combinar componentes para resolver problemas. É como em uma máquina: cada elemento tem que fazer bem sua parte especializada para que todas as partes, de forma coordenada, executem uma atividade.

Um exemplo é a estratégia de acesso a dados em diferentes soluções de arquitetura. Algumas arquiteturas podem combinar o componente de acesso ao banco

de dados com os componentes de representação de entidades, e fazer com que todas as chamadas ao banco de dados retornem, obrigatoriamente, objetos representativos de entidades; neste caso, temos um componente de acesso aos dados que é composto de três componentes: um de acesso ao banco de dados, um com o grupo de Entidades da Regra de Negócio, e uma terceira, que faz a adaptação dos dados que vem do banco de dados e os transforma nas entidades. E ainda que internamente este componente seja fortemente acoplado, ele funciona de forma desacoplada do resto do sistema, e ainda assim é possível injetar nele um outro componente de acesso a dados, e fazer com que este componente funcione de forma mais desacoplada.

Experiência: usar o mínimo

Um dos conselhos que podemos indicar, no uso de componentes, é sempre usar o mínimo necessário e, sempre que possível, compor componentes em novos componentes para entregar comportamentos esperados. Usando a composição de componentes, podemos coordenar o uso dos componentes existentes de acordo com uma regra que definimos, e não precisamos reconstruir parte do software, usando unidades de código que já reconhecemos como seguras e bem implementadas.

Experiência: adaptar a ferramenta à necessidade

Um dos maiores erros que encontramos no uso de componentes na construção de arquiteturas é a adaptação da arquitetura aos componentes. Muitas vezes, por estarmos familiarizado com algumas soluções ou uso de componentes, decidimos que um grupo de componentes ou decisões de arquitetura deve ser usado para resolver um problema, e começamos a desenhar a arquitetura usando estes componentes e decisões; aí, com o tempo, começamos a perceber que a solução tende a ficar mais complexa com o uso desses componentes, e começamos a “entortar” a arquitetura para que ela use os componentes desejados.

Isso cria inúmeras adaptações na arquitetura para que a solução funcione de uma forma não otimizada, e cria um grande débito técnico para o futuro. O ideal é desenhar a arquitetura básica para entender o problema a resolver, e então começar a identificar os componentes a usar, e a partir daí tomar as decisões técnicas.

Um exemplo comum dessa abordagem é a escolha de um framework específico para a implementação de alguns componentes da aplicação antes de entender se a solução precisa desse framework. A quantidade de código necessária para fazer com que a adaptação funcione é muitas vezes maior do que a quantidade de código necessária para fazer a solução funcionar.

Experiência: conhecer arquiteturas diferentes

E como aplicar as melhores ideias de arquitetura e de componentização? Existem duas formas conhecidas: praticando e estudando. Com a prática na criação de arquiteturas e implementação dessas arquiteturas, começamos a aprender o que fazer e o que não, baseando-se na qualidade do que fazemos. Uma arquitetura ruim pode ser analisada e melhorada de forma iterativa, ou seja, depois de algumas revisões. Por isso, o pensamento ágil é importante nesse momento: faça, teste, analise e melhore.

Com tantos projetos de software abertos hoje em dia, temos a possibilidade de ler e aprender com o que outras pessoas fizeram. Vídeos e livros também podem ajudar, mas ler códigos e ver uma arquitetura implementada, entendendo as razões de cada decisão, tem um efeito semelhante ao da prática: você está em contato com algo que funciona de verdade, e que tem qualidades e defeitos.

Só com a experiência é possível criar uma arquitetura de alta qualidade, e experiência vem com o aprendizado prático, fazendo, corrigindo e melhorando.

O QUE VOCÊ VIU NESTA AULA?

A organização da Arquitetura de um software em componentes nos ajuda a entender a solução em pedaços de código e o relacionamento entre cada um destes pedaços. Mas a criação dos componentes pode ser desafiadora se não entendermos como criar componentes de alta qualidade, se não entendermos como os componentes devem se comportar dentro das camadas e como devem ser usados de forma a resolver os problemas.

Criar uma Arquitetura usando componentes é desafiador porque exige conhecer os diferentes tipos de componentes existentes, conhecer as limitações de cada um desses componentes, e decidir quando criar um componente ou quando usar um componente que já existe.

O que achou da aula? No Discord, você pode conversar com os seus colegas sobre o que aprendeu aqui! E caso tenha alguma dúvida, estamos sempre disponíveis na comunidade para te ajudar! Te esperamos!

REFERÊNCIAS

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [s.l.]: Addison-Wesley Professional, 2003.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

THE Architecture of Open-Source Applications. Disponível em: <<https://aosabook.org/en/>>. Acesso em: 01 jun. 2023.

USE case. Disponível em: <<https://martinfowler.com/bliki/UseCase.html>>. Acesso em: 01 jun. 2023.

PALAVRAS-CHAVE

Camadas. Casos de uso. Arquitetura limpa.

EXEMPLO

POS TECH